友元

友元类

```
#include<iostream>
#include<string>
using namespace std;

class building;//声明类

class goodgay{
public:
    goodgay();//构造函数

    void visit();

    building *pbuilding;//指针的类型决定了它所指向的内存空间的数据类型

    ~goodgay();

};

class building{

    friend class goodgay;//类前加上friend，定义该类可以访问此类中private

public:
    building();//构造函数

    string m_livingroom;

private:
    string m_bedroom;
};

//类外写成员函数
building::building()
{
    m_livingroom="客厅";
    m_bedroom="卧室";
}

goodgay::goodgay()
{
    pbuilding = new building;
}

void goodgay::visit()//返回值 类::函数()
{
    cout<<pbuilding->m_livingroom<<endl;
    cout<<pbuilding->m_bedroom<<endl;
}
```

```
goodgay::~goodgay()
{
    delete pbuilding;
}

void test1()
{
    goodgay gg;
    gg.visit();
}

int main()
{
    test1();

    return 0;
}
```

全局函数作为友元

```
#include<iostream>
#include<string>
using namespace std;

class building{

    friend void goodgay(building &building);//加上friend，定义该函数可以访问类中
private

public:
    building()
    {
        m_livingroom="客厅";
        m_bedroom="卧室";
    }

    string m_livingroom;

private:
    string m_bedroom;
};

void goodgay(building &building)
{
    cout<<building.m_livingroom<<endl;
    cout<<building.m_bedroom<<endl;
}

void test1()
{
    building building;
    goodgay(building);
```

```
}

int main()
{
    test1();

    return 0;
}
```

成员函数作为友元

```
#include<iostream>
#include<string>
using namespace std;

class building;

class goodgay{
public:
    goodgay();

    void visit1();//让visit函数可以访问building中私有成员

    void visit2();//让visit函数不可以访问building中的私有成员

    building *pbuilding;
};

class building{
public:

    friend void goodgay::visit1();//goodgay类中成员函数前加上友元，并且说明是哪个类，就可以访问building类中的private

    building();

    string m_livingroom;

private:
    string m_bedroom;

};

building::building()
{
    m_livingroom="客厅";
    m_bedroom="卧室";
}

goodgay::goodgay()
{
    pbuilding=new building;
}
```

```
void goodgay::visit1()
{
    cout<<pbuilding->m_livingroom<<endl;
    cout<<pbuilding->m_bedroom<<endl;
}

void goodgay::visit2()
{
    cout<<pbuilding->m_livingroom<<endl;
//  cout<<pbuilding->m_bedroom<<endl;
}

void test1()
{
    goodgay gg;
    gg.visit1();
    gg.visit2();
}

int main()
{
    test1();

    return 0;
}
```